Experiment Details

IndicSeq2Seq: Hindi-to-Marathi Translation

---

1. Environment and Hardware

Platform: Kaggle Notebooks
Compute: Dual NVIDIA T4 GPUs (15 GB VRAM per GPU)
Framework: PyTorch (with DataParallel for multi-GPU training)

Key Libraries:
- `torch`, `torchtext`: Model architecture and data loading
- `sentencepiece`: Subword tokenization
- `sacrebleu`: BLEU and CHRF metric calculation
- `pandas`, `numpy`: Data manipulation

---

2. Dataset Setup

The parallel Hindi-Marathi corpus was sourced from public Indic NLP datasets.

Data Splits:
- **Training Set:** ~200,000 sentence pairs
- **Validation Set:** ~10,000 sentence pairs
- **Qualitative Test Set:** 45 curated sentences for error analysis

Preprocessing:
- Filtered out empty strings and pairs where the source-to-target length ratio exceeded 3.0.
- Removed duplicate pairs to prevent data leakage.
- Trained a joint SentencePiece BPE tokenizer with a vocabulary size of 32,000 tokens.
- Applied max sequence length of 150 tokens during training.

---

3. Hyperparameters

Hyperparameter	Value

Batch Size	64
Max Sequence Length	150 tokens
Embedding Dimension	256
Hidden Dimension	512
Encoder Layers	2 (Bidirectional LSTM)
Decoder Layers	2 (Unidirectional LSTM)
Dropout Rate	0.3
Teacher Forcing Ratio	0.5
Optimizer	Adam
Learning Rate (Epochs 1-15)	3e-4
Learning Rate (Epoch 16)	1.5e-4 (Continuation Training)
Gradient Clipping	1.0 (L2 norm)

---

4. Execution Steps

To reproduce the training and evaluation:

1. **Environment Setup:** Ensure the environment has dual GPUs available and install requirements.
2. **Tokenizer Training:** Run the SentencePiece training script on the combined Hindi and Marathi corpus to generate the `bpe_indic_32k.model` file.
3. **Training Execution:**
   - Execute the training loop using the `DataParallel` wrapper.
   - The model checkpoints are saved at the end of each epoch.
4. **Continuation Training:**
   - Load the checkpoint from Epoch 13 (best validation loss).
   - Halve the learning rate to `1.5e-4` and train for 3 additional epochs (Epochs 14-16).
5. **Inference & Evaluation:**
   - Load the Epoch 16 checkpoint.

- Run the evaluation script using **Beam Search (K=4)** and **Repetition Blocking** to generate the final translations.
- Calculate Corpus BLEU and CHRF scores using the `sacrebleu` library.

---

5. Inference Configurations

Several decoding strategies were experimented with during inference. The final selected strategy was:

- **Algorithm:** Beam Search
- **Beam Width (K):** 4
- **Length Penalty (Alpha):** 0.6
- **Repetition Blocking:**
- Subtracted a penalty of 1.5 from the logits of tokens that were already generated in the current output sequence.
- Hard-blocked consecutive repeating bigrams.

This configuration provided the best balance between BLEU score improvement and grammatical fluency, effectively mitigating the repetitive degeneration issues commonly seen in standard greedy decoding.